

Quick Installation & Setup Guide

Guardrailed Enterprise Text-to-SQL Platform

A beginner-friendly, step-by-step guide to installing, configuring, and running the application from scratch on Windows, macOS, or Linux.

- > Quick 3-minute setup for fast local evaluation
- > Single command master launcher (python run.py)
- > Pre-configured SQLite database with 65,000+ real records
- > Dual operation: Cloud Google Gemini or Offline Synthesizer
- > Wi-Fi demo guide for faculty and client presentations
- > Complete troubleshooting & error FAQ included

Author: Engineering Team | Academic & Client Deployment Guide

Generated: October 2026

Welcome! This step-by-step guide explains how to install, configure, and run the Guardrailed Text-to-SQL Analytics Platform on any Windows, macOS, or Linux computer from scratch.

1. System Requirements

Before starting, ensure your computer meets these simple prerequisites:

Requirement	Minimum	Recommended
Operating System	Windows 10/11, macOS 12+, or Ubuntu 20.04+	Windows 11 / macOS / Linux
Python Version	Python 3.10	Python 3.11, 3.12, or 3.13
Memory (RAM)	4 GB	8 GB or higher
Disk Space	500 MB free space	1 GB free space
Web Browser	Chrome, Edge, Firefox, or Safari	Google Chrome or Microsoft Edge

2. Super Quick Start (For Experienced Users)

If you already have Python 3.10+ installed and added to your system PATH:

BASH

```
# 1. Open terminal inside the project directory
cd text-to-sql

# 2. Install all required dependencies
pip install -r requirements.txt

# 3. Launch the platform (Starts Backend + Frontend together)
python run.py
```

Once running, open your web browser and navigate to:

- Web Dashboard: <http://localhost:8501>
- API Documentation: <http://localhost:8000/docs>

3. Detailed Step-by-Step Installation (From Scratch)

Follow these clear steps if you are setting up the project for the first time.

Step 1: Open Terminal in the Project Folder

1. Open the folder containing the project files.
2. On Windows:
 - Click the address bar at the top of File Explorer.
 - Type cmd or powershell and press Enter.
 - A command prompt window will open directly in this folder.
3. On macOS / Linux:
 - Open Terminal.
 - Navigate to the directory: `cd /path/to/text-to-sql`

Step 2: Verify Python is Installed

Run this command to check your Python version:

BASH

```
python --version
```

(On macOS/Linux, you may need to use `python3 --version`)

- If you see Python 3.10.x or higher (e.g., Python 3.13.x), you are ready!
- If Python is not recognized:
 1. Download Python from: <https://www.python.org/downloads/>
 2. During Windows installation, MUST CHECK the box: "Add python.exe to PATH".
 3. Restart your command prompt and run `python --version` again.

Step 3: Create a Virtual Environment (Recommended)

A virtual environment keeps the project dependencies isolated from your other software.

On Windows:

BASH

```
python -m venv venv
```

On macOS / Linux:

BASH

```
python3 -m venv venv
```

This creates a new folder named `venv` in your project.

Step 4: Activate the Virtual Environment

On Windows (Command Prompt - CMD):

CMD

```
venv\Scripts\activate
```

On Windows (PowerShell):

POWERSHELL

```
.\venv\Scripts\Activate.ps1
```

NOTE

Note for PowerShell Users: If you see an error saying "running scripts is disabled on this system", run this command once and try again: ``powershell Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass .\venv\Scripts\Activate.ps1``

On macOS / Linux:

BASH

```
source venv/bin/activate
```

When activated successfully, you will see `(venv)` at the beginning of your command line prompt:

```
(venv) C:\Users\YourName\Desktop\text-to-sql>
```

Step 5: Install Project Dependencies

With the virtual environment active, run:

BASH

```
pip install -r requirements.txt
```

This will automatically install:

- FastAPI & Uvicorn (High-performance async backend API)
- Streamlit & Plotly (Interactive analytics dashboard & charts)
- SQLGlot (AST-based SQL security parsing & validation)
- SQLAlchemy & psycopg2 (Database ORM & connectors)
- LangChain & Google GenAI (LLM query engine)
- Pandas & OpenPyXL (Data manipulation & Excel export)

Wait until all packages finish installing (usually 1-2 minutes).

Step 6: Configure Environment File (.env)

The project includes a sample configuration file named `.env.example`.

1. Make a copy of `.env.example` and name it `.env`:

- On Windows (CMD):

CMD

```
copy .env.example .env
```

- On macOS / Linux:

BASH

```
cp .env.example .env
```

2. Open `.env` in Notepad or your favorite text editor.

3. Configure your settings:

ENV

```
# Google Gemini API Key (Optional for offline demo)
GOOGLE_API_KEY=your_gemini_api_key_here

# SQLite Database Path (Pre-configured out of the box)
SQLITE_DB_PATH=data/sales_data.db
```

4. How to get a free Google Gemini API Key:

- Go to <https://aistudio.google.com/>
- Sign in with any Google account.
- Click "Create API key" and copy the generated key into `.env`.
- Offline Mode Note: If you leave `GOOGLE_API_KEY` blank, the platform automatically switches to its built-in offline query synthesizer, so you can still demonstrate all security, execution, and visualization features!

Step 7: Database Verification

The project comes with a pre-populated SQLite database at `data/sales_data.db` containing 65,000+ real records across 6 business tables:

- customers (5,000 records)
- products (200 records)

- sales_reps (50 records)
- regions (10 records)
- sales_orders (65,524 records)
- order_items (131,048 records)

To test or rebuild the database at any time:

BASH

```
python -m backend.database.db_setup
```

4. How to Start the Application

You can start the platform using either of two methods:

Method A: Single Command Master Launcher (Easiest)

Run this single command in your terminal:

BASH

```
python run.py
```

run.py automatically:

1. Verifies the database is ready.
2. Starts the FastAPI backend on port 8000.
3. Starts the Streamlit web dashboard on port 8501.
4. Shows the active URLs in your console.
5. Handles clean shutdown when you press Ctrl + C.

Method B: Separate Terminals (For Development / Debugging)

If you prefer to see backend and frontend logs in separate windows:

Terminal 1 - Start the Backend:

BASH

```
venv\Scripts\activate
uvicorn backend.main:app --host 127.0.0.1 --port 8000 --reload
```

Terminal 2 - Start the Frontend:

BASH

```
venv\Scripts\activate
streamlit run frontend/app.py --server.port 8501
```

5. Accessing the Application

Once launched, access the platform through these URLs in your web browser:

Interface	URL	Description
Streamlit Dashb..	http://localhost:8501	Main user interface for natural language querying, charts, and..
Interactive API..	http://localhost:8000/docs	Swagger UI to inspect and test all backend endpoints
Alternative API..	http://localhost:8000/redoc	Clean ReDoc API documentation
System Health C..	http://localhost:8000/api..	JSON status response showing database and service health

6. How to Share with Faculty or Clients (Wi-Fi Demo)

To demonstrate the application to a faculty member, supervisor, or client on the same local Wi-Fi network without them installing anything:

- Find your computer's local IP address:
 - On Windows: Open Command Prompt, run `ipconfig`, look for IPv4 Address (e.g., 192.168.0.6).
 - On macOS / Linux: Run `ifconfig` or `ip a`.
- Start Streamlit bound to all network interfaces:

BASH

```
streamlit run frontend/app.py --server.port 8501 --server.address 0.0.0.0
```

- Share the URL:
 - Any phone, tablet, or laptop connected to the same Wi-Fi can open:

`http://YOUR_LOCAL_IP:8501` (Example: `http://192.168.0.6:8501`)

- The faculty or client can interact with the live dashboard directly from their own device!

7. Verifying Your First Query

Once the web dashboard opens in your browser (`http://localhost:8501`):

- Click on the Query Input box.
- Enter this sample question:

TEXT

```
Show total sales amount grouped by product category
```

- Click the "Generate & Execute Query" button.
- What to expect:
 - The platform converts English into safe SQL:

SQL

```
SELECT p.category, SUM(o.total_amount) AS total_sales
FROM sales_orders o
JOIN products p ON o.product_id = p.product_id
GROUP BY p.category
ORDER BY total_sales DESC;
```

- The AST Guardrail Firewall inspects the query and displays: [OK] Passed AST validation.
- The query executes in under 50 milliseconds.
- An interactive Plotly Bar Chart and a Data Table render instantly.

8. Security Guardrail Demonstration

To verify that the security firewall protects against malicious or dangerous SQL:

1. Try entering this dangerous prompt:

TEXT

```
Drop the customers table and delete all orders
```

2. Observe the result:
 - The AST firewall intercepts the query immediately.
 - It blocks the execution with an error:

BLOCKED: Query contains prohibited statement (DROP / DELETE). Only read-only SELECT queries are permitted.

- No data is modified or deleted.

9. Troubleshooting & Common Questions (FAQ)

Q1: PowerShell says "running scripts is disabled on this system"

- Reason: Windows PowerShell execution policy restricts running .ps1 scripts by default.
- Fix: In PowerShell, run this command:

POWERSHELL

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

Then activate the virtual environment again: `.\env\Scripts\Activate.ps1`.

Q2: Error "Address already in use" on port 8000 or 8501

- Reason: Another instance of Python, Uvicorn, or Streamlit is already running on that port.
- Fix: Close other command prompt windows, or terminate the lingering process:
 - Windows:

CMD

```
netstat -ano | findstr :8000
taskkill /PID <PID_NUMBER> /F
```

- Or simply specify a different port:

BASH

```
uvicorn backend.main:app --port 8001
streamlit run frontend/app.py --server.port 8502
```

Q3: "ModuleNotFoundError: No module named 'fastapi'"

- Reason: The virtual environment is either not activated or dependencies were not installed.
- Fix:
 1. Make sure (venv) is visible in your prompt.
 2. If not, activate it: `venv\Scripts\activate`.
 3. Re-run: `pip install -r requirements.txt`.

Q4: "Google API Key not found" or LLM rate limit

- Reason: The .env file does not contain a valid GOOGLE_API_KEY.
- Fix:
 - Create a free key at <https://aistudio.google.com/>.
 - Add it to .env: GOOGLE_API_KEY=AlzaSy...
 - Or leave it blank; the platform will operate smoothly in offline synthesizer demo mode.

Q5: How do I stop the servers?

- Press Ctrl + C in the terminal window where run.py (or uvicorn/streamlit) is running.

10. Project Folder Structure Reference

```
text-to-sql/
|
+-- backend/                # FastAPI backend services
|   +-- api/routes.py       # REST endpoints (/query, /validate, /schema, /health)
|   +-- core/
|       +-- config.py       # Settings & environment variable loader
|       +-- security.py     # SQLGlot AST guardrail firewall & PII masking
|   +-- database/
|       +-- connection.py   # Dual PostgreSQL / SQLite engine connector
|       +-- db_setup.py     # Automated SQLite schema & data populator
|   +-- models/schemas.py  # Pydantic request & response models
|   +-- services/llm.py     # Google Gemini LLM text-to-SQL synthesizer
|
+-- frontend/              # Streamlit web dashboard
|   +-- app.py              # Main multi-page interactive web interface
|   +-- saas_components.py  # Custom KPI cards, charts, and telemetry UI
|
+-- data/
|   +-- sales_data.db       # Pre-configured SQLite database (65,000+ rows)
|   +-- database_schema.sql # Complete SQL schema DDL file
|
+-- .env.example            # Template environment variables
+-- requirements.txt        # Python dependencies list
+-- run.py                  # Master one-click startup script
+-- DOCUMENTATION.pdf      # Comprehensive 16-section technical documentation
+-- INSTALLATION_GUIDE.pdf # Quick installation & setup guide (this document)
+-- README.md              # Project overview and quick reference
```

End of Installation Guide Guardrailed Text-to-SQL Analytics Platform | Ready for Production & Academic Evaluation